Reusable Designated Verifier NIZK from Lossy Trapdoor Functions

Riddhi Ghosal
Nokia Bell Labs

Ilan Komargodski
The Hebrew University of Jerusalem

Brent Waters
UT Austin and NTT Research

Non-Interactive Zero Knowledge

Prover

holds x and witness w

$\pi \leftarrow P(\text{crs}, x, w)$

proves that $x \in L$

NP language $L = \{x : \exists w, (x, w) \in R\}$

π



Verifier

holds only x

$V(\text{crs}, x, \pi) \in \{0, 1\}$

accepts iff output is 1

Completeness	*Honest proofs of true statements always verify*
Soundness	*No efficient prover gets false statements accepted*
Zero knowledge	$\exists$ PPT S, $\forall (x, w) \in R : (\text{crs}, P(\text{crs}, x, w)) \approx_c S(x)$ *π reveals nothing beyond the fact that $x \in L$*

Constructing NIZKs for NP

Random Oracle Model

Fiat–Shamir transform [FS86, PS96]

Constructing NIZKs for NP

Random Oracle Model

Fiat–Shamir transform [FS86, PS96]

CRS Model

QR / factoring [BFM88, DMP87, BDMP91, FLS90], **LWE** [CCH+19, PS19], **Pairings** [GOS06], **LPN + MQ** [DJJ24], **LPN + DDH** [BKM20] and **DDH** [JJ21].

Constructing NIZKs for NP

Random Oracle Model

Fiat–Shamir transform [FS86, PS96]

CRS Model

QR / factoring [BFM88, DMP87, BDMP91, FLS90], **LWE** [CCH+19, PS19], **Pairings** [GOS06], **LPN + MQ** [DJJ24], **LPN + DDH** [BKM20] and **DDH** [JJ21].

Constructions for generic primitives

- **Doubly enhanced Trapdoor Permutation** [BFM88, SMP87, FLS90]
- **Circular Secure FHE** [CCH+19]
- **Strong KDM Encryption** [CCRR18]
- **SNARGs** [KMY20]
- **iO** [SW14]
- **BARGs** [CW23]

Constructing NIZKs for NP

Random Oracle Model

Fiat–Shamir transform [FS86, PS96]

CRS Model

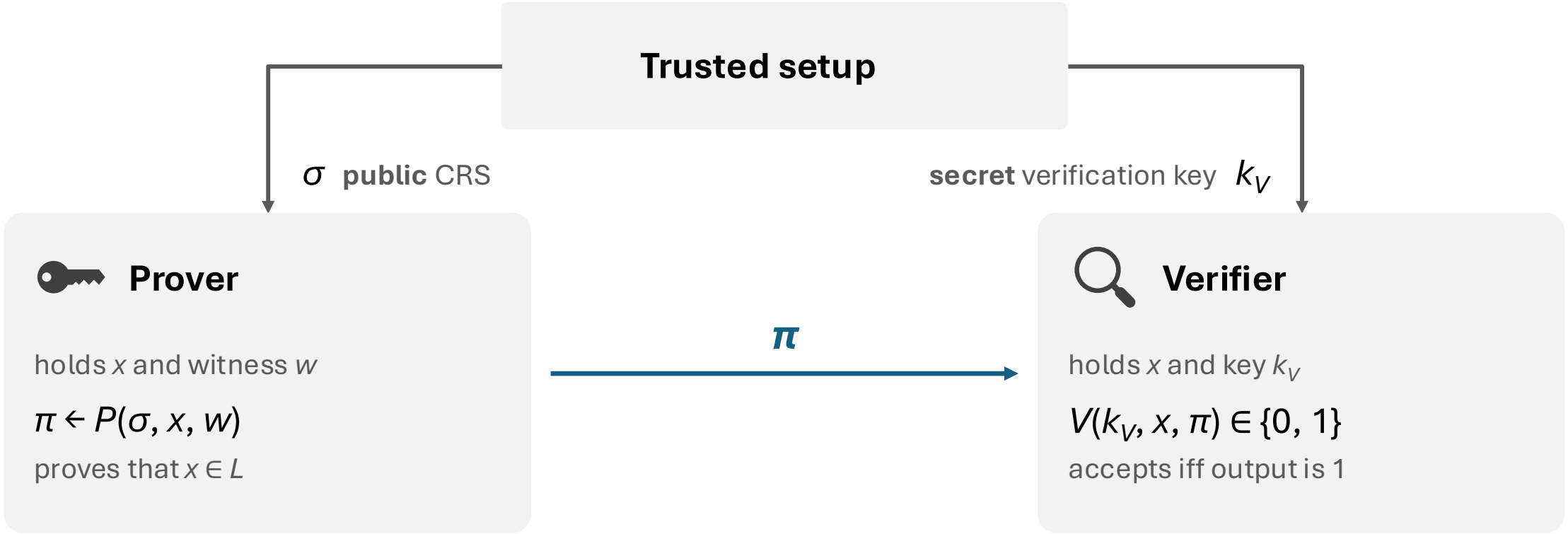
QR / factoring [BFM88, DMP87, BDMP91, FLS90], **LWE** [CCH+19, PS19], **Pairings** [GOS06], **LPN + MQ** [DJJ24], **LPN + DDH** [BKM20] and **DDH** [JJ21].

Constructions for generic primitives

- **Doubly enhanced Trapdoor Permutation** [BFM88, SMP87, FLS90]
- **Circular Secure FHE** [CCH+19]
- **Strong KDM Encryption** [CCRR18]
- **SNARGs** [KMY20]
- **iO** [SW14]
- **BARGs** [CW23]

Questions: *What is the minimal assumption necessary to construct NIZK? Where does NIZK lie in the hierarchy of cryptographic primitives?*

Designated Verifier NIZK



Reusable Soundness Soundness holds even if the prover can generate multiple proofs and test whether the verifier accepts.

Why Designated Verifier NIZK?

A relaxation of NIZK

Few generic assumptions imply DV-NIZK but not NIZK:

PKE + Circular Secure SKE [LQRWW19]

CPA secure PKE [PSV06]*

*non-reusable soundness

Why Designated Verifier NIZK?

A relaxation of NIZK

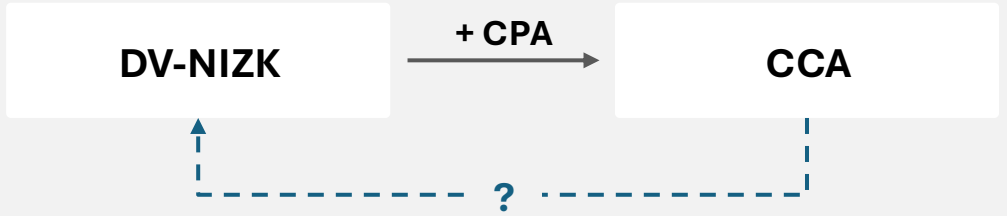
Few generic assumptions imply DV-NIZK but not NIZK:

PKE + Circular Secure SKE [LQRWW19]

CPA secure PKE [PSV06]*

Connection to CCA security

- Suffices to construct CCA security
- Shares similar techniques with CCA



*non-reusable soundness

Why Designated Verifier NIZK?

A relaxation of NIZK

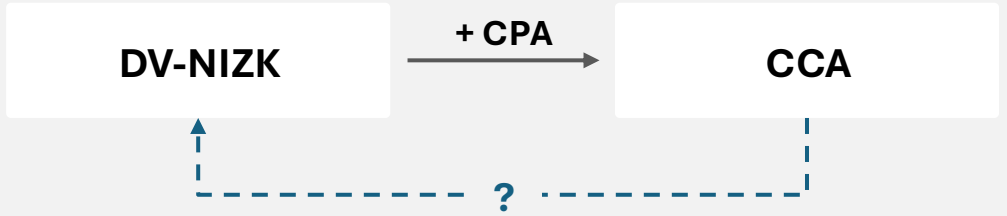
Few generic assumptions imply DV-NIZK but not NIZK:

PKE + Circular Secure SKE [LQRWW19]

CPA secure PKE [PSV06]*

Connection to CCA security

- Suffices to construct CCA security
- Shares similar techniques with CCA



In this work, our goal is to improve understanding of where this object lies relative to NIZKs and CCA security.

*non-reusable soundness

Why Designated Verifier NIZK?

A relaxation of NIZK

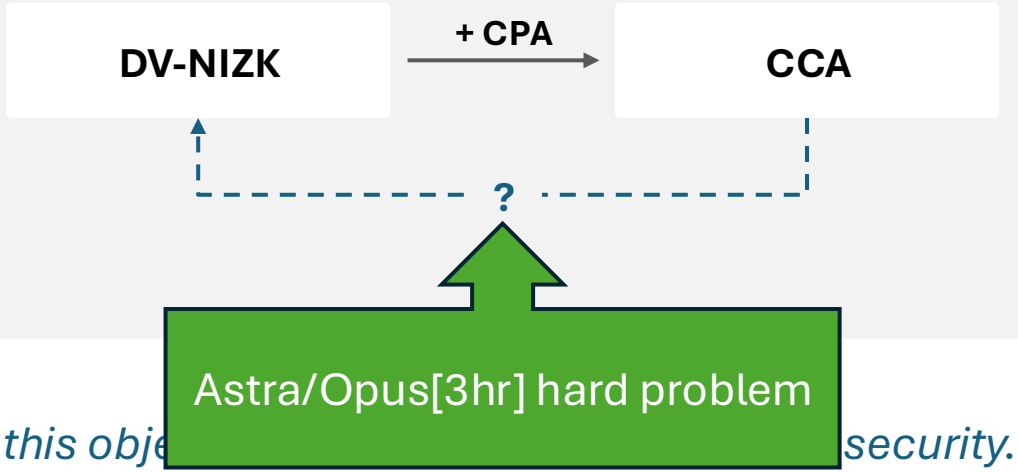
Few generic assumptions imply DV-NIZK but not NIZK:

PKE + Circular Secure SKE [LQRWW19]

CPA secure PKE [PSV06]*

Connection to CCA security

- Suffices to construct CCA security
- Shares similar techniques with CCA



In this work, our goal is to improve understanding of where this obje security.

*non-reusable soundness

Generic construction of Designated Verifier NIZK

Prior work [LQRWW19]



Generic construction of Designated Verifier NIZK

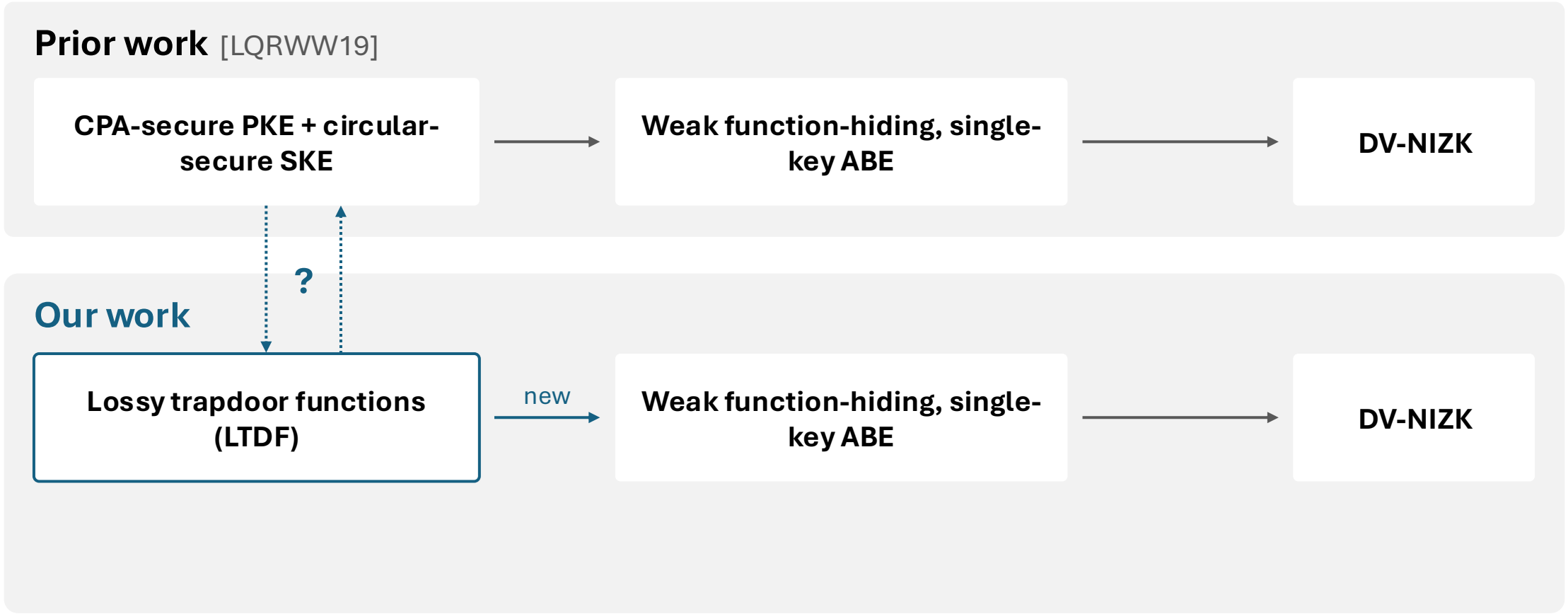
Prior work [LQRWW19]



Our work



Generic construction of Designated Verifier NIZK



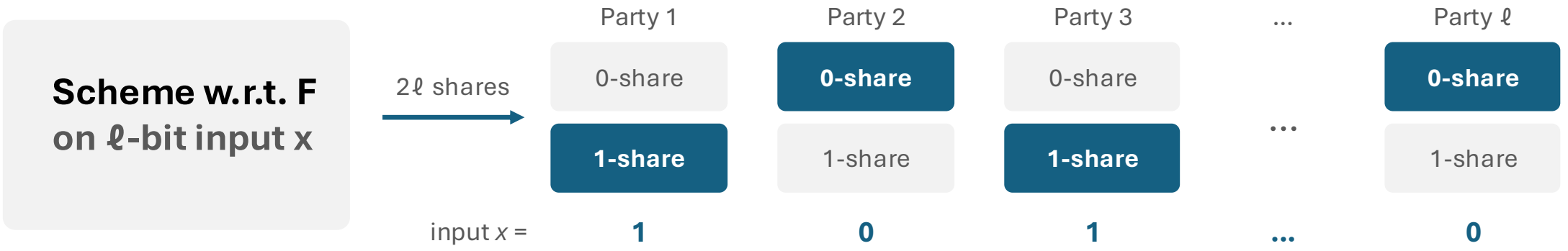
Main Technical contribution

Randomness-recoverable non-monotone secret sharing scheme

Scheme w.r.t. F
on ℓ-bit input x

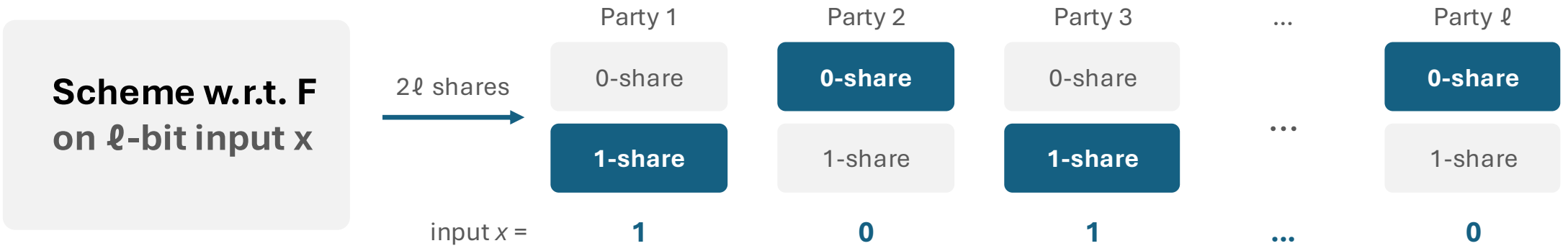
Main Technical contribution

Randomness-recoverable non-monotone secret sharing scheme



Main Technical contribution

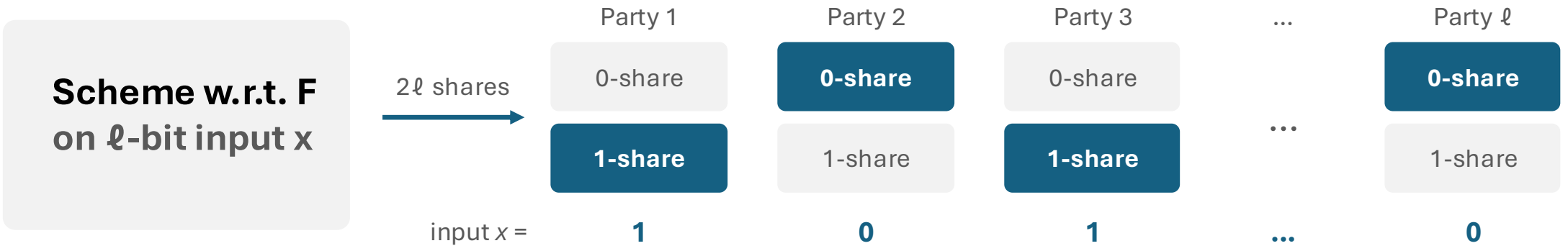
Randomness-recoverable non-monotone secret sharing scheme



Correctness	If x is **authorized** ($F(x) = 1$), the selected shares can *reconstruct* the secret s
Privacy	If x is **unauthorized** ($F(x) = 0$), the selected shares *reveal nothing* about s

Main Technical contribution

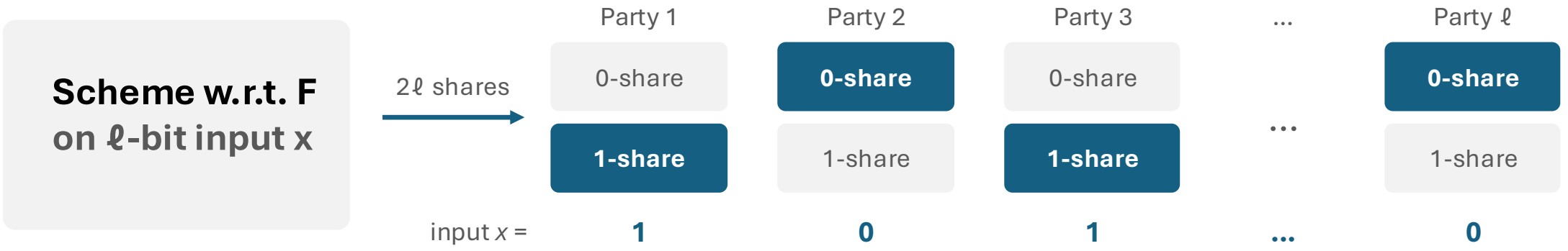
Randomness-recoverable non-monotone secret sharing scheme



Correctness	If x is **authorized** ($F(x) = 1$), the selected shares can *reconstruct* the secret s
Privacy	If x is **unauthorized** ($F(x) = 0$), the selected shares *reveal nothing* about s
Randomness recovery	Any ℓ *authorized* shares can *regenerate* all 2ℓ shares

Main Technical contribution

Randomness-recoverable non-monotone secret sharing scheme



Correctness

Privacy

We construct unconditionally secure, randomness-recoverable, non-monotone secret sharing for PBP

Randomness recovery Any ℓ *authorized* shares can *regenerate* all 2ℓ shares

Secret Sharing Scheme



Secret Sharing Scheme



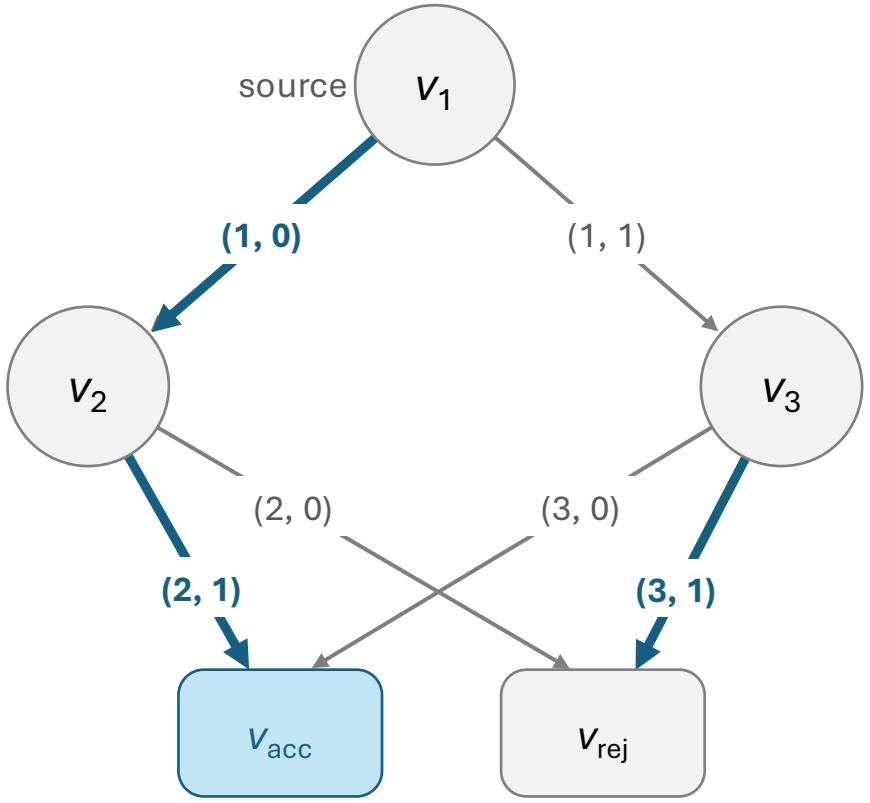
- DAG with source v_1 and sinks v_{acc}, v_{rej}

Secret Sharing Scheme



- DAG with source v_1 and sinks v_{acc}, v_{rej}
- Each v_i, $i \in [\ell]$, has out-edges $(i, 0)$, $(i, 1)$

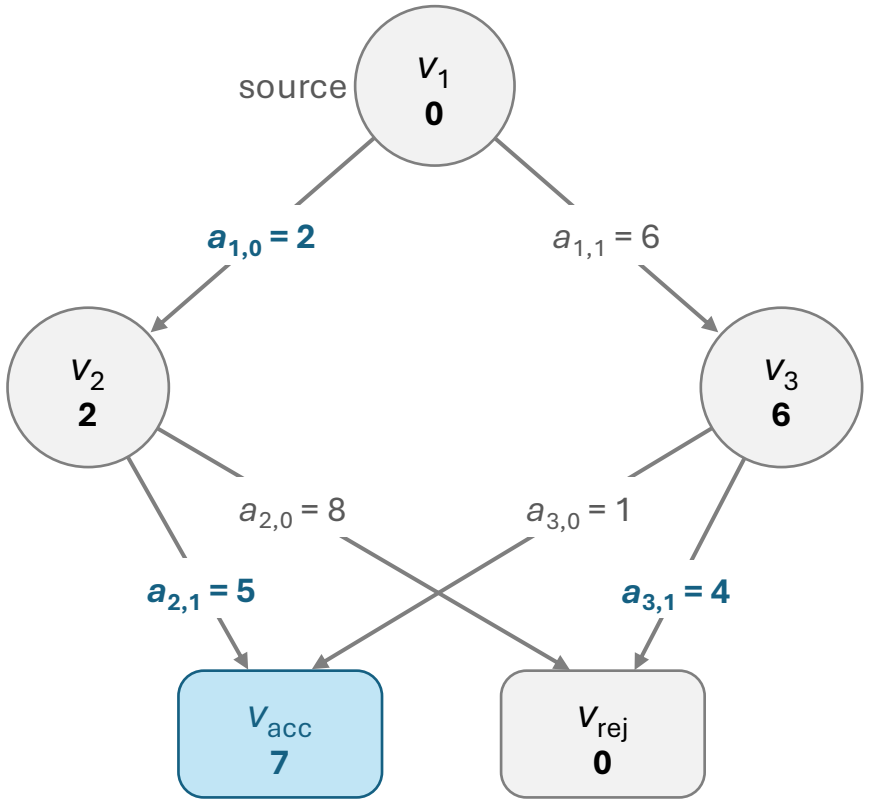
Secret Sharing Scheme



$F \in$ PBP → **Branching program G** of size $\ell = \text{poly}(n)$

- DAG with source v_1 and sinks v_{acc}, v_{rej}
- Each v_i, $i \in [\ell]$, has out-edges $(i, 0)$, $(i, 1)$
- Edges $\{(i, x_i)\}$ form a path to v_{acc} if $F(x) = 1$, to v_{rej} if $F(x) = 0$

Share assignment for secret *s*



Secret $s = 7$

Party i holds $a_{i,0}$ and $a_{i,1}$

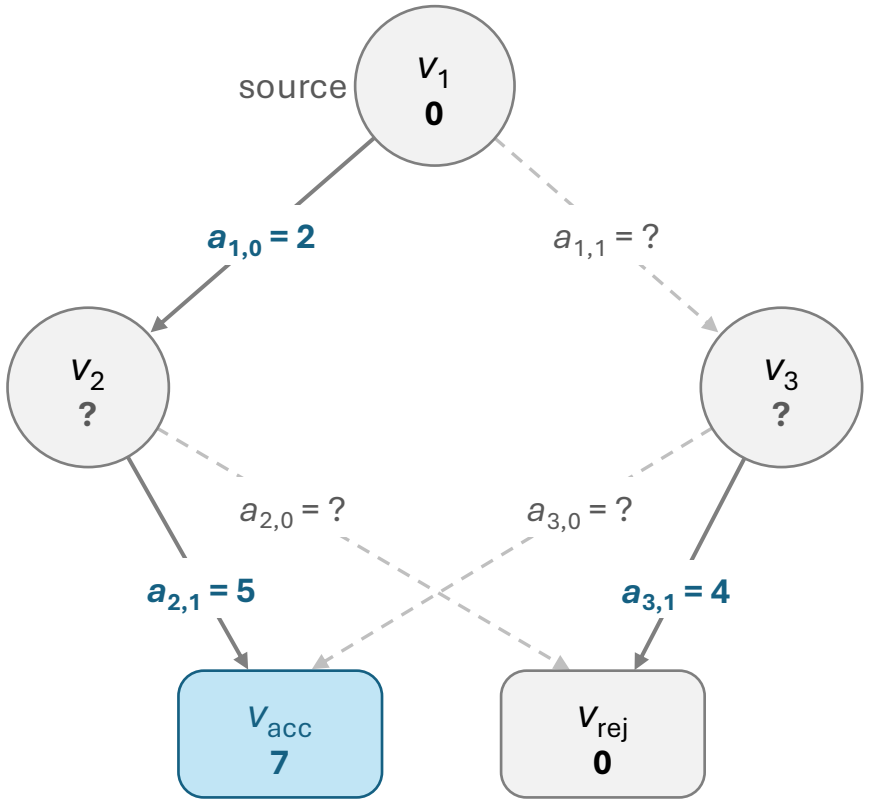
Share assignment

- $v_{acc} \leftarrow s$; v_1, $v_{rej} \leftarrow 0$
- Other nodes ← uniform random
- Edge share = head value − tail value

Arithmetic mod 10, for illustration

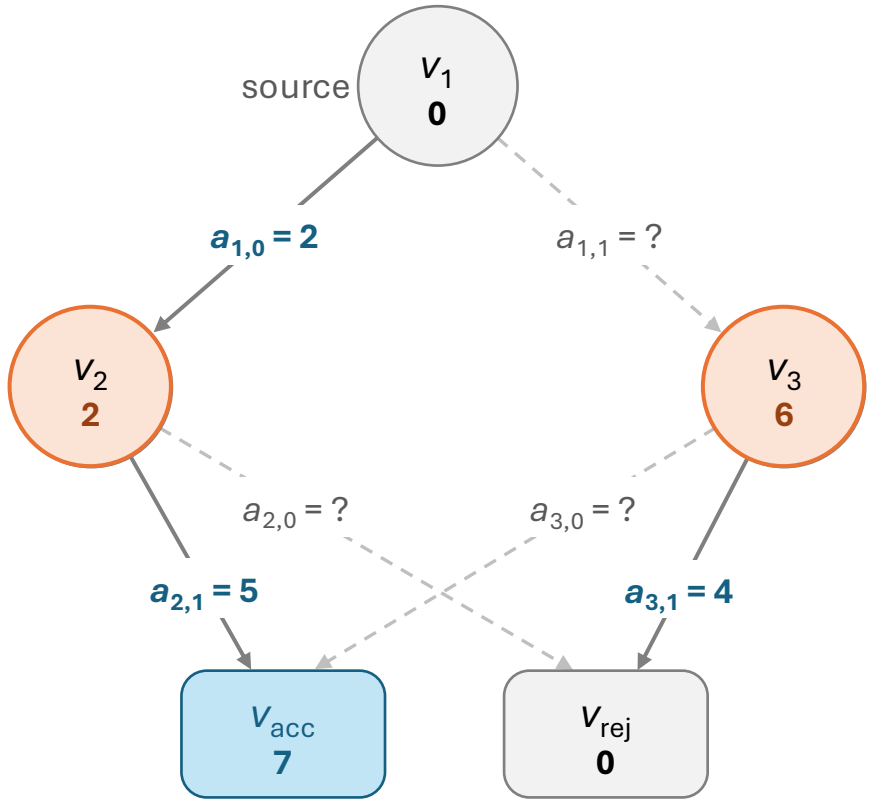
Share regeneration

Input $x = (0, 1, 1)$
known: s, $a_{1,0}$, $a_{2,1}$, $a_{3,1}$



For any x, the held shares cover one out-edge of every non-sink node. Arithmetic mod 10

Share regeneration



Input $x = (0, 1, 1)$

known: s, $a_{1,0}$, $a_{2,1}$, $a_{3,1}$

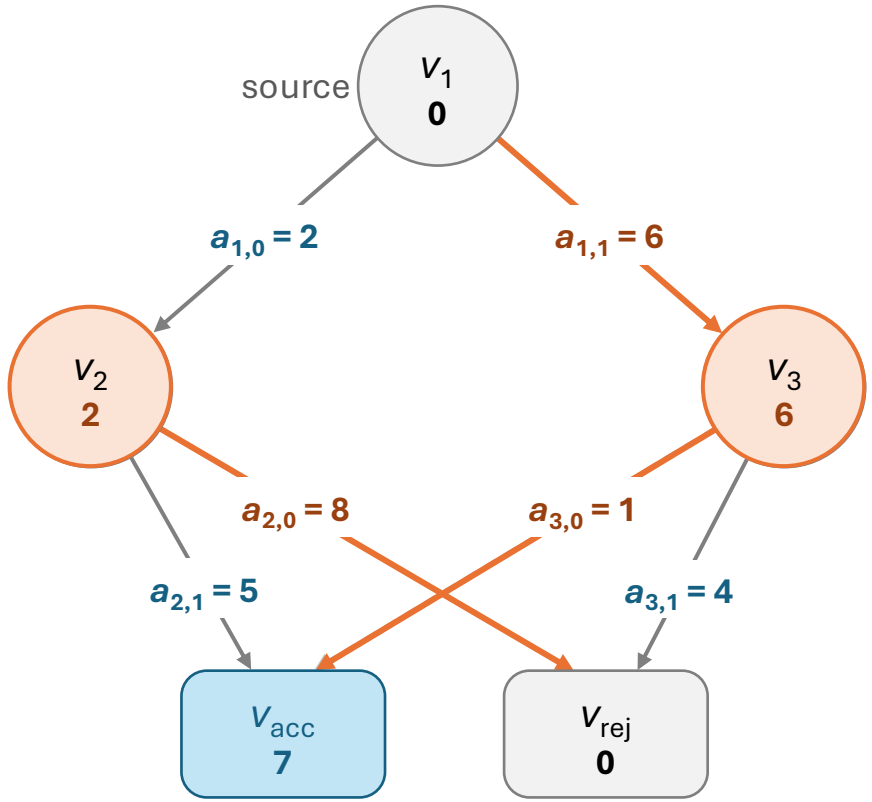
1 Recover node values

$v_2 = s - a_{2,1} = 7 - 5 = 2$

$v_3 = 0 - a_{3,1} = 0 - 4 = 6$

For any x, the held shares cover one out-edge of every non-sink node. Arithmetic mod 10

Share regeneration



Input $x = (0, 1, 1)$

known: s, $a_{1,0}$, $a_{2,1}$, $a_{3,1}$

1 Recover node values

$v_2 = s - a_{2,1} = 7 - 5 = 2$

$v_3 = 0 - a_{3,1} = 0 - 4 = 6$

2 Recompute missing shares

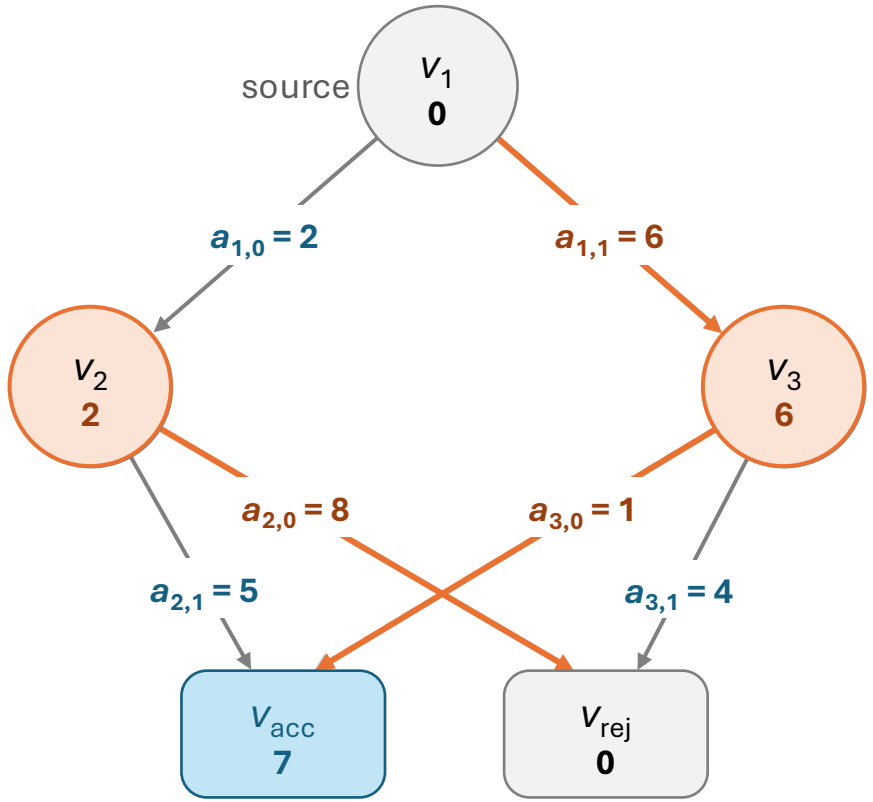
$a_{1,1} = v_3 - v_1 = 6 - 0 = 6$

$a_{2,0} = v_{rej} - v_2 = 0 - 2 = 8$

$a_{3,0} = v_{acc} - v_3 = 7 - 6 = 1$

For any x, the held shares cover one out-edge of every non-sink node. Arithmetic mod 10

Share regeneration



Input $x = (0, 1, 1)$

known: s, $a_{1,0}$, $a_{2,1}$, $a_{3,1}$

1 Recover node values

$v_2 = s - a_{2,1} = 7 - 5 = 2$

$v_3 = 0 - a_{3,1} = 0 - 4 = 6$

2 Recompute missing shares

$a_{1,1} = v_3 - v_1 = 6 - 0 = 6$

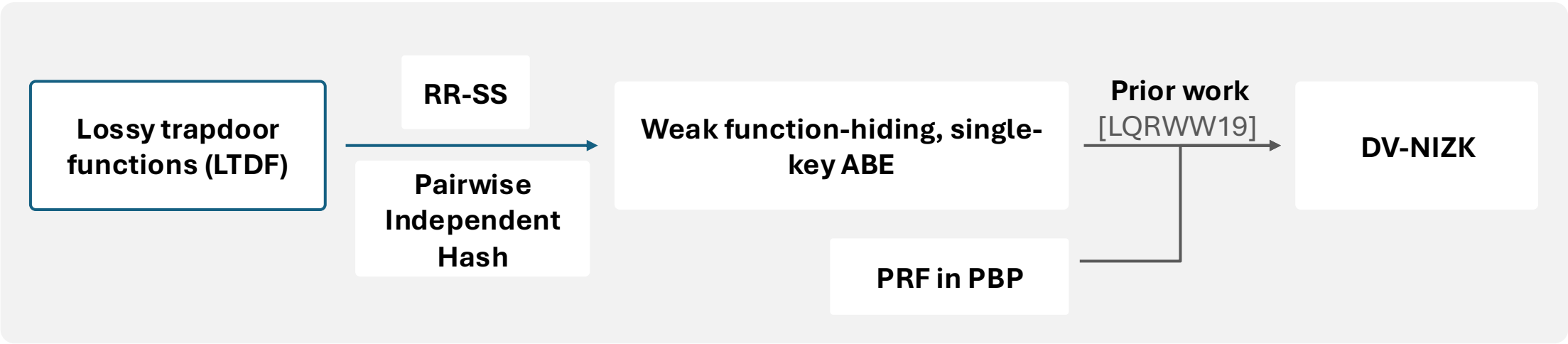
$a_{2,0} = v_{rej} - v_2 = 0 - 2 = 8$

$a_{3,0} = v_{acc} - v_3 = 7 - 6 = 1$

We also construct randomness-recoverable LSSS from OWF.

For any x, the held shares cover one out-edge of every non-sink node. Arithmetic mod 10

Summary


Lossy trapdoor functions (LTDF)
RR-SS
Pairwise Independent Hash
Weak function-hiding, single-key ABE
Prior work
[LQRWW19]
PRF in PBP
DV-NIZK

Open Problems

1. **DV-NIZK from CCA / TDF / CPA?**
   Or a separation?

Open Problems

1. **DV-NIZK from CCA / TDF / CPA?**
   Or a separation?

2. **NIZK from (malicious) DV-NIZK?**
   Or a separation?

Open Problems

1 **DV-NIZK from CCA / TDF / CPA?**
Or a separation?

2 **NIZK from (malicious) DV-NIZK?**
Or a separation?

3 **One-time sound DV-NIZK vs. NIZK**

One-time sound DV-NIZK
separated from NIZK

CPA
separated from NIZK

Open Problems

1 **DV-NIZK from CCA / TDF / CPA?**
Or a separation?

2 **NIZK from (malicious) DV-NIZK?**
Or a separation?

3 **One-time sound DV-NIZK vs. NIZK**

One-time sound DV-NIZK
separated from NIZK

CPA
separated from NIZK

4 **Other applications for randomness-recoverable secret sharing**

Thank You!

Attribute-Based Encryption (ABE)

[SW05, GPSW06]

Public-key ABE: a key for f decrypts a ciphertext with attribute x only if $f(x) = 1$

Ciphertext

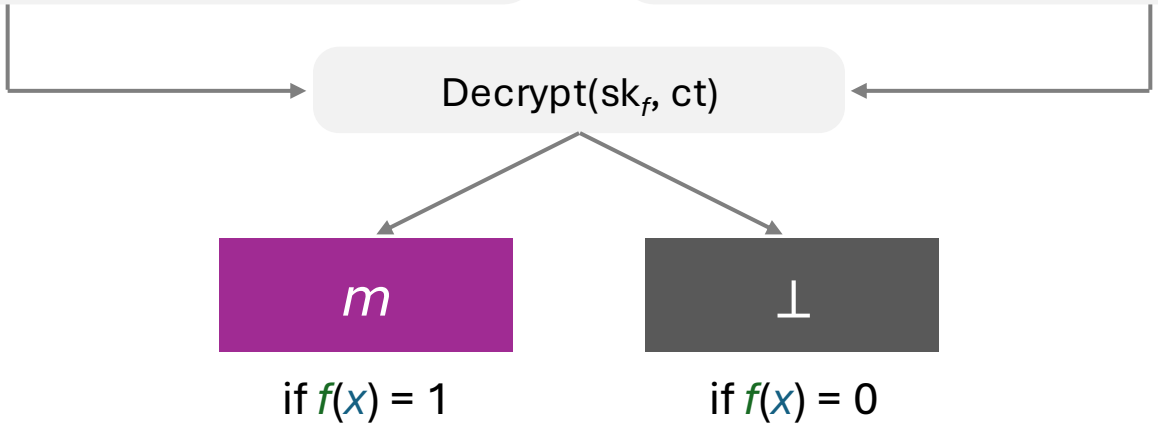


public attribute $x \in X$, message m

Secret key



function $f: X \to \{0, 1\}$



Semantic security: ct hides m if $f(x) = 0$

Weak function-hiding ABE

Real experiment



Simulated experiment



Construction overview

Setup	Sample a pairwise-independent hash H
Keygen ($f \in \{0,1\}^{\ell}$)	1. Generate $(ev_{i,0}, td_{i,1})$ for all $i \in [\ell]$ 2. Let the pk be all eval keys 3. Let msk be all trapdoors, and sk_f be $\{f, td_{i,f_i}\}$
Enc (pk, $x \in \{0,1\}^{\lambda}$, $m \in \{0,1\}^{\gamma}$)	1. Sample random secret s 2. Split into 2ℓ shares using RR-SS 3. Ciphertext has LTDF evaluations of all shares and $m \oplus H(s)$
Dec (pp, sk, x, ct)	1. If $f(x) = 0$, output $\perp$ 2. Invert the ℓ ciphertexts using the corresponding trapdoors 3. Regenerate all 2ℓ shares 4. Reconstruct secret s 5. Check that all LTDF evals match the given ciphertexts 6. Output $ct_0 \oplus H(s')$